Informe Actividad Pre-Parcial

Parte 1: Operaciones básicas con archivos

Al empezar, el programa comprueba si hay un archivo llamado estudiantes.txt. Si no lo encuentra, lo crea con unos cuantos registros de ejemplo. Después abre el archivo y lee todo, mostrando cada línea; esto es justo como funciona el acceso secuencial.

```
¡Hola! Este programa trabaja con un archivo llamado estudiantes.txt.
No encontré el archivo, así que lo creé con algunos registros de ejemplo.

Contenido del archivo
Estos son los estudiantes registrados:
Nombre,Edad,Nota
Ana,21,89
Luis,23,92
Marta,20,95
Carlos,22,85
```

Parte 2: ¿Cuándo usar acceso secuencial y cuándo acceso directo?

A la hora de manejar archivos, no es lo mismo leerlos de un extremo a otro que ir directo a una posición. Si nuestra idea es procesar todo el contenido —listar cada estudiante, sacar un promedio o buscar algo sin saber dónde está— lo natural es el acceso secuencial.

Ahora bien, si ya sabemos cuál registro buscamos, el acceso directo nos evita leer el resto del archivo. Por ejemplo, si el usuario pide ver la línea 3, simplemente vamos allí y listo; muy útil cuando el archivo es grande y queremos respuestas puntuales.

Resumen: secuencial para todo, directo para lo puntual.

```java
// PARTE 2 - Punto 2: Acceso directo — muestra una línea específica del archivo por número

    static void mostrarLineaDirecta(File file, int num) throws IOException {
        BufferedReader r = new BufferedReader(new FileReader(file));
        String line;
        int cur = 1;
        boolean found = false;
        while ((line = r.readLine()) != null) {
            if (cur++ == num) {
                System.out.println("Línea " + num + ": " + line);
                found = true;
                break;
            }
        }
        r.close();
        if (!found) System.out.println("Número de línea fuera de rango.");
    }

```

```
¿Qué línea te gustaría ver? Ingresa el número: 3
Línea 3: Luis,23,92
Gracias por usar la consulta directa.
```

Parte 3: ¿En qué se diferencian la mezcla directa y la mezcla natural?

Antes de ordenar datos de forma externa conviene entender dos aproximaciones. Las dos usan bloques ordenados, pero las tratan distinto.

La mezcla directa toma el arreglo, lo corta en trozos de un mismo tamaño (1, 2, 4, 8,… elementos) y, par a par, los funde. Es sencilla y transparente, pero no le importa si los datos ya vienen casi ordenados: igual recorre todo, haciendo pasadas extra sin necesidad.

La mezcla natural intenta ser más lista: no corta en bloques rígidos, sino que busca "corridas" que ya están ordenadas de por sí. Esas corridas se fusionan entre sí, y si la entrada ya tiene partes ordenadas, el proceso se hace más rápido porque hay menos mezclas que hacer.

El programa incluye “mezclaDirecta” para fusionar dos bloques y `corridasNaturales` para detectar corridas en un arreglo, demostrando ambas ideas.

```java
// PARTE 3 - Punto 4: Mezcla directa — fusiona dos bloques ya ordenados
static int[] mezclaDirecta(int[] a, int[] b) {
    int[] res = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length)
        res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < a.length) res[k++] = a[i++];
    while (j < b.length) res[k++] = b[j++];
    return res;
}

// PARTE 3 - Punto 5: Detección de corridas naturales (secuencias ya ordenadas)
static List<int[]> corridasNaturales(int[] arr) {
    List<int[]> corridas = new ArrayList<>();
    int inicio = 0;
    while (inicio < arr.length) {
        int fin = inicio + 1;
        while (fin < arr.length && arr[fin] >= arr[fin - 1]) fin++;
        corridas.add(Arrays.copyOfRange(arr, inicio, fin));
        inicio = fin;
    }
    return corridas;
}

```

```java
// PARTE 3 - Punto 4: Mezcla directa
    int[] b1 = {10, 30, 50, 70}, b2 = {20, 40, 60, 80};
    System.out.println("\n  Ahora hacemos una mezcla directa con dos bloques  ");
    System.out.println("Bloque 1: " + Arrays.toString(b1));
    System.out.println("Bloque 2: " + Arrays.toString(b2));
    System.out.println("La combinación ordenada queda así: " + Arrays.toString(mezclaDirecta(b1, b2)));

    // PARTE 3 - Punto 5: Corridas naturales
    int[] arr = {3, 5, 7, 2, 4, 8, 1, 6};
    List<int[]> corridas = corridasNaturales(arr);
    System.out.println("\n  Buscando corridas naturales en " + Arrays.toString(arr) + "  ");
    for (int i = 0; i < corridas.size(); i++)
        System.out.println("  Corrida " + (i + 1) + ": " + Arrays.toString(corridas.get(i)));
```

```
  Ahora hacemos una mezcla directa con dos bloques
Bloque 1: [10, 30, 50, 70]
Bloque 2: [20, 40, 60, 80]
La combinación ordenada queda así: [10, 20, 30, 40, 50, 60, 70, 80]

  Buscando corridas naturales en [3, 5, 7, 2, 4, 8, 1, 6]
  Corrida 1: [3, 5, 7]
  Corrida 2: [2, 4, 8]
  Corrida 3: [1, 6]
```

Parte 4: Recursividad para contar estudiantes con nota >= 90

La función "contarNotasAltas" mira el primer estudiante, suma uno si tiene nota ≥ 90 y luego se llama a sí misma con la lista restante.

¿Por qué recurrir a la recursividad? Porque deja el código claro y directo: el conteo es 1 o 0 según la nota del primer estudiante, más el conteo del resto. No hace falta manejar índices ni variables extra.

En Java, cada llamada recursiva usa espacio en la pila. Si la lista fuese muy larga (miles o millones de entradas) podríamos quemar la pila y obtener un StackOverflowError. Java tampoco optimiza la recursión por cola, así que no hay truco para esquivar ese gasto. Para unos pocos estudiantes está bien, pero en escenarios reales quizá convenga un bucle sencillo.

```java

// PARTE 4 - Punto 7: Recursividad — cuenta estudiantes con nota >= 90

static int contarNotasAltas(List<String[]> lista, int i) {
    if (i == lista.size()) return 0;
    int nota = Integer.parseInt(lista.get(i)[2].trim());
    return (nota >= 90 ? 1 : 0) + contarNotasAltas(lista, i + 1);
}

```

```java

// PARTE 4 - Punto 7: Contar recursivamente (análisis en informe.md)
System.out.println("\n  Contando con recursividad  ");
System.out.println("Aquí vemos cuántos estudiantes tienen nota de 90 o más:");
System.out.println("Total: " + contarNotasAltas(estudiantes, 0));

sc.close();
```

```
  Contando con recursividad
Aquí vemos cuántos estudiantes tienen nota de 90 o más:
Total: 2
PS C:\Users\Usuario\Desktop\Julian\Workspace\ucompensar>
```